MTF: an Open-Source Metamorphic Testing Framework for LLM-based systems

Theis Henry
Universite Grenoble Alpes
Grenoble, France
theis.henry@etu.univ-grenoble-alpes.fr

Sian Savourat
Universite Grenoble Alpes
Grenoble, France
Sian.Savourat@etu.univ-grenoble-alpes.fr

Lydie du Bousquet
Universite Grenoble Alpes
Grenoble, France
lydie.du-bousquet@univ-grenoble-alpes.fr

Masahide Nakamura
Center of Mathematical and Data Sciences
Kobe University
Kobe, Japan
masa-n@people.kobe-u.ac.jp

Abstract

Metamorphic testing is an emerging approach to solve oracle problems. It relies on relations among different executions of the System Under Test (SUT), called metamorphic relations. This article presents an open-source metamorphic testing framework dedicated to validating LLM-based web services applications. It is decomposed into a metamorphic test generator and a test driver. Both rely on a predefined set of metamorphic relations implemented as libraries that can be completed.

Keywords

Metamorphic Testing, Framework, LLMs

ACM Reference Format:
Theis Henry, Sian Savourat, Lydie du Bousquet, and Masahide Nakamura. 2025. MTF: an Open-Source Metamorphic Testing Framework for LLM-based systems. In *2025 5th International Conference on Artificial Intelligence and Application Technologies (AIAT 2025), December 04–06, 2025, Kyoto, Japan.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3787120.3787123

1 Introduction

The rapid evolution of Large Language Models (LLMs) has generated significant interest in incorporating them into a wide array of software applications. However, building and evaluating LLM-powered systems remains complex [4]. The unpredictable nature of LLM outputs (which are inherently non-deterministic) makes validation difficult to manage. A recent analysis of open-source LLM projects identified low-quality and inaccurate responses as major issues [1].

Because Metamorphic Testing is known to be an interesting alleviation to the oracle problem, it has been applied to test LLM projects. Metamorphic Testing (MT) has been introduced by T.Y. Chen [3]. It relies on multiple executions of different inputs in order to check an expected relation among the outputs, instead of checking the output correctness for each execution. These relations are called Metamorphic Relations (MRs). They are used to derive follow-up test cases from original ones.

Even if the MT principle is simple, it is quite difficult to identify relevant MRs relations. Moreover, there are not so many tools that apply the approach to automate the test generation and/or the test execution. For this reason, we propose an open-source MT framework that includes a test generator and a test-driver dedicated to LLM-based web service validation. It includes classical MRs dedicated to the validation of LLM-oriented applications. Those MRs are implemented as libraries that can be easily completed by the users to fit their needs. The framework has been experimented with to test several web services based on LLMs.

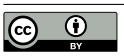
We first introduce MT and related work in Section 2. In Section 3, we present the developed framework. We illustrate the use of the framework on a case study detailed in Section 4, before concluding and discussing perspectives.

2 Related work

Metamorphic Testing (MT) has become more and more popular in scenarios where test oracles are absent or hard to define [17]. To generate tests, MT makes use of Metamorphic Relations (MRs) that are expected to hold between multiple inputs and the related output. An MR is often expressed as an *input pattern* (R_i) and an *output pattern* (R_o). The input pattern is applied to a (set of) original test cases to produce a set of *follow-up test inputs*. The output pattern is used to check the results; it takes as input the reference output (y_r) and the observed output (y_o) and returns a verdict. For example, if the System Under Test (SUT) is the sine function *y=sin(x)*, one can consider the *input pattern* $R_i(x) : (x + j * 2 * \Pi)$, $j \in \mathbb{Z}^*$ and the related output pattern $R_o(y_r, y_o) : y_r == y_o$.

The automation of MT follow-up test generation and execution is often done by a dedicated program built specifically for experimentation. In this article, we propose an open-source framework for LLM-based applications that includes and extends the set of MRs proposed in the literature, using character, word and sentence level categorizations [9, 10, 13, 19].

Our proposition is inspired by other existing MT frameworks. GeMTest is a general MT framework in Python that uses Pytest as test-driver [18]. Like our framework, it is open-source. It is not dedicated to LLM-based application validation. Metal is dedicated



This work is licensed under a Creative Commons Attribution 4.0 International License.
AIAT 2025, Kyoto, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2290-5/25/12
https://doi.org/10.1145/3787120.3787123

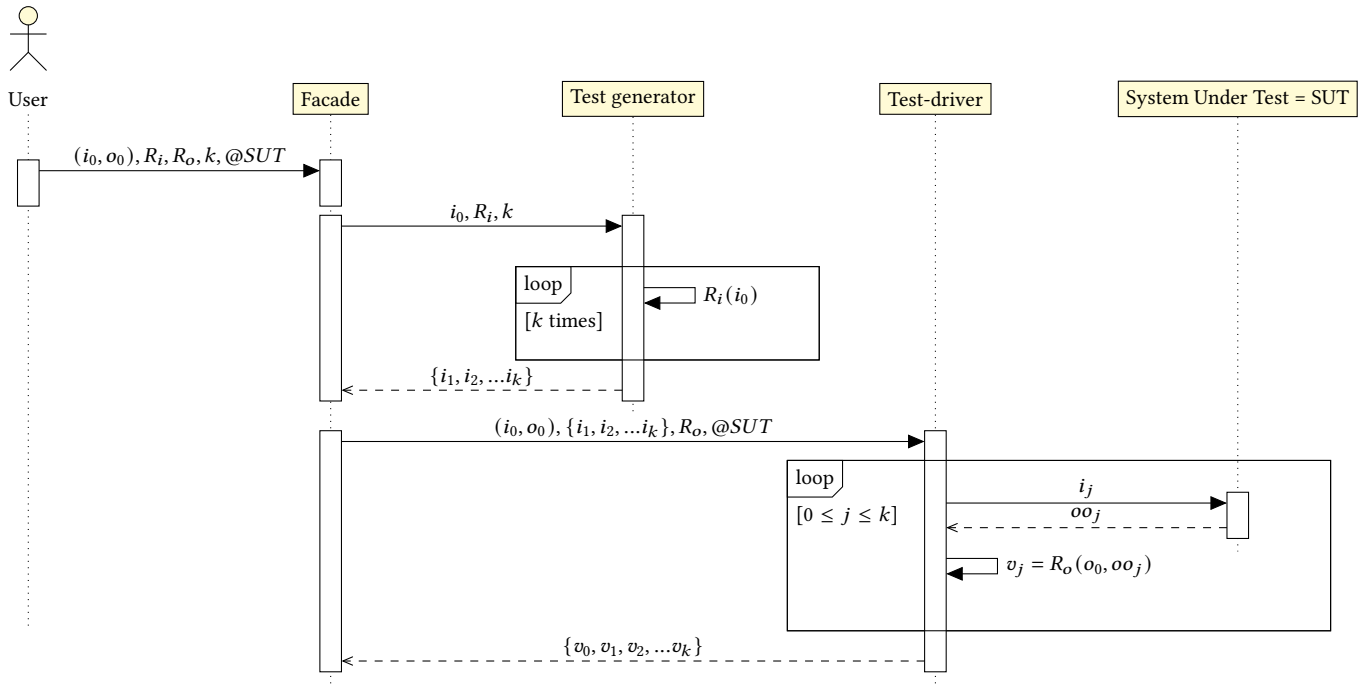


Figure 1: MTF basic usage use-case

to LLM quality analysis [10], and Drowzee to fact-conflicting hallucination detection [11].

The JFuzz tool proposed by Zhu [22] in 2015 relies on the xUnit frameworks. The output patterns are implemented as annotations. MT4ImgRec is a MT framework dedicated to image recognition software [2]. RMT is a MT framework dedicated to autonomous driving, composed of three parts among which are a test generator and a metamorphic relation validator. The first part is a NLP-based semantic parser that infers MR from human-written scenarios [5]. MetaExploreX supports visualization and data exploration capabilities, in order to reduce manual analysis and to improve MR selection [6]. Gotten is a model-driven tool that automates the construction of MT environments [7].

With the same objective to test LLM-based applications, the authors of [21] explore adaptive testing to strategically select diverse inputs to ensure an even distribution across the input space. The oracle function is not detailed in this article.

3 Framework

MTF is a framework built to apply MT to a large number of Systems Under Test (SUT), and specifically focused on those with LLM functionalities. The framework is composed of two main parts: the test input generator and the test-driver. Both are coordinated by a facade component (see Fig. 1).

The test input generator. takes as input (1) the original test case (i_0, o_0), (2) the chosen input pattern R_i, (3) the length of the follow-up test set $k \in \mathbb{N}^*$. It produces a (set of) follow-up test inputs $\{i_1, i_2, ...i_k\}$, in various formats such as list, JSON file, ... The testers can use an input pattern provided within the framework, or they can define their own ones in Python.

Currently, the framework proposes 19 text-oriented input pattern implementations, listed in the appendix. They are derived from 11 input patterns defined in [20] and [12]. They allow for rephrasing an input while preserving its meaning. This includes relations such as substituting a word with its synonym, modifying sentence length, or altering the tone or style of the speaker. The different implementations use both ChatGPT and BERT, and include adaptations to Japanese language specificities. Our framework also proposes 14 specific input pattern implementations derived for our experimentation (detailed in the next section).

The test-driver. is responsible for the test execution. It takes as input (1) the original test case(s) (i_0, o_0), (2) the follow-up test inputs $\{i_1, i_2, ...i_k\}$, (3) the output pattern R_o and (4) the path to the System Under Test (*SUT*). The test-driver executes the *SUT* with each input $\{i_0, i_1, ...i_k\}$, collects the observed outputs $\{oo_0, oo_1, oo_2, ...oo_k\}$ and applies the output pattern R_o to each of them. It returns a set of verdicts $\{v_0, v_1, ...v_k\}$ ...

In the MT literature, equality is an output pattern frequently used [17]. But strict equality is not relevant for text output comparison. Similarity analysis to evaluate whether the meanings of two sentences are equivalent is more relevant. Our framework currently proposes three output pattern implementations to assess semantic similarity with BERT and ChatGPT and a word-based statistical metric.

The facade. component orchestrates the collaboration between the test generator and the test-driver as illustrated in Fig. 1. The framework proposes two facade components. A simple version is a command-line script. The second one is a user interface based on a web service technology.

Our framework is open-source and can be downloaded here [8]. Currently, it is available with input patterns and output patterns for LLM-based application validation. However, it is built to support other types of systems under test. Any user can add input or output pattern implementation, for instance, in Python, as long as it respects the input and output format.

(a) Original input (i_0): "How many countries are there in the world?"
(b) Original output (o_0): "Hmm, let's see. There are many countries in the world. To keep it simple, we say there are currently 193. Each has its own culture, language, cuisine, and it's really interesting! By the way, Japan is one of them."
(c) Chosen input Pattern (R_i): @Chat-GPT: "reformulate the sentence changing one word with its synonym"
(d) One follow-up test produced with R_i (i_1): "How many nations are there in the world?"
(e) A second follow-up test produced with R_i (i_2): "How many states are there in the world?"
(f) Observed output for i_1 (oo_1): "Well, there are currently about 193 countries in the world. It's an impressive number, isn't it? Did you understand?"
(g) Observed output for i_2 (oo_2): "Well, at the moment, there are about 193 countries in the world, according to the most recent data. But this number can sometimes increase or decrease a bit. So, there is no really definitive number."
(h) Chosen output Pattern ($R_o^{Sim-GPT}$): "Chat-GPT text similarity function score $\geq$ 75%"
(i) Verdict obtained with $R_o^{Sim-GPT}(o_0, oo_1)$ (v_1): True (because the similarity score $\approx$ 80%)
(j) Verdict obtained with $R_o^{Sim-GPT}(o_0, oo_2)$ (v_2): True (because the similarity score $\approx$ 85%)
(k) Manually modified output (o'_2): "Well, at the moment, there are about 200 countries in the world, according to the most recent data. But this number can sometimes increase or decrease a bit. So, there is no really definitive number."
(l) Similarity scores obtained for $R_o(o_0, o'2)$: with $R_o^{Sim-GPT} \approx 70\%$ with $R_o^{Sim-Bert}$ (sentenceBert) $\approx 23\%$
(m) Similarity scores obtained for $R_o(o_2, o'2)$: with $R_o^{Sim-GPT} \approx 90\%$ with $R_o^{Sim-Bert}$ (sentenceBert) $\approx 14\%$

Figure 2: An example of test case generation, execution, and validation

4 Framework validation process

In order to validate our proposition, we apply the framework to a case study, and we elaborated a process dedicated to the validation of LLM-oriented input and output patterns.

4.1 Case study

Mei-chan is a virtual agent, designed to interact with old people or isolated persons. The first versions of Mei-chan were almost completely scripted; open questions were manually analyzed. Additional web services built around chatGPT prompts were progressively added to Mei-chan in order to process natural language requests.

Mei-chan's current version integrates LLM-based services to diversify the dialogue and the usages. In the following, we focus on the "Need Companion" function of Mei-chan. It aims at capturing the user needs in order to achieve different tasks such as agenda, messaging, music playing, information request, ...[14–16].

Mei-chan uses the "ELI5" service, which gives a simple answer to a complex question as if a child was expressing it. The service treats only natural language for the question in input and the answer in output. It is expected to produce similar outputs for inputs that have the same meaning.

To check that, we applied different input patterns that introduced variations in phrasing without altering semantic content. These transformations include sentence length adjustments, rephrasing using synonym substitution, and modifications in speaker tone or demographic style.

For example, Figures 2(a) and 2(b) present an original question (i_0) and an answer (o_0) obtained using the "ELI5" service (considered as correct). Let's apply twice our input pattern R_i that asks ChatGPT to reformulate a sentence, changing one word into a synonym 2(c). We obtain the two different follow-up inputs i_1 and i_2, given in 2(d) and 2(e) thanks to our test generator. The test-driver submits them to the "ELI5" service (SUT) and obtains outputs oo_1 and oo_2, given Fig. 2(f) and 2(g).

As it can be noticed, o_0, oo_1 and oo_2 answers are quite different, even if the numerical value is the same. In order to check the similarity of the follow-up test case outputs with the original one, we have two output patterns relying respectively on ChatGPT and on BERT to evaluate the similarity of two texts. The user can define a threshold of 75% under which the texts are supposed to be different (see 2(h)). ChatGPT gives a similarity score between o_0 and oo_1 around 80% and a similarity between o_0 and oo_2 around 85%. Note that the similarity score returned by ChatGPT can differ from one execution to another due to the non-deterministic behaviour of the LLM engine.

4.2 Validation process

The validation process means to check that (1) the framework works as expected and (2) the input/output patterns proposed in the framework satisfy their specifications.

The first part was done through several case studies, among which the Mei-chan agent was presented previously. During this study, we added dedicated input and output patterns, which demonstrated how easily the framework can be extended.

The second part has to be done for each pattern. To test an *input pattern*, we need to examine the results manually to decide if it satisfies its specification. For instance, to test an input pattern that replaces a word with a synonym, you have to check that the result keeps the same sense as the original formulation.

Fig. 3 displays the results we obtained on a subset of input relations. Column 2 indicates the number of cases that were manually examined. Columns 3 and 4 indicate how many of them were declared to be conformed with the input relation specification (Pass) or not (Failed). We used Japanese sentences for the library intended for that language, and inputs in English, French, and Japanese for the multilingual one.

As it can be noticed, the *word_substitution_synonym with ChatGPT*) pattern has a bad satisfaction score. The defaults occur when the language used for the prompt (English) is different from the language used in the input (French or Japanese), probably because the text is translated twice (e.g., from French/Japanese to English and then from English to French/Japanese) to achieve the operation. It reveals how it is important to propose input patterns dedicated to each language for semantic operations.

The *output patterns* are intended to check that two texts in natural language have the same meaning. To test them, we execute them and compare the returned similarity scores with the ones computed manually. For example, we ask several persons to estimate the similarity of texts given in Fig. 2(i) and (j) with 2(g). If the difference was less than 15%, we consider that the test passed; it failed otherwise. For instance, we transformed the value 193 into 200 to obtain the text given in Fig. 2(k), and evaluated the similarity scores with the output patterns 2(l) and (m). It can be seen that ChatGPT and sentenceBERT produce very different scores for this example.

To achieve the validation of the *output patterns*, we built two sets of tests. The first test set is composed of 100 tests obtained during the Mei-chan validation process. The second test set is composed of 20 edge cases manually created. The results are displayed in Table 1. As shown in the table, both models achieved accuracy rates close to 100%, with the embedding-based method performing slightly better overall. In the more specific edge cases, performance was lower, but ChatGPT showed marginally better results in those scenarios.

Overall, the results remain strong and demonstrate the potential and reliability of these comparison functions in the context of metamorphic testing.

5 Conclusion and perspectives

In this article, we presented MTF, an open-source framework designed to apply metamorphic testing methods to software systems[1]. Currently, it includes MR dedicated to the validation of programs that incorporate Large Language Models (LLMs) in their execution.

We detailed the framework's architecture and explained how to develop metamorphic relations to build reusable libraries compatible with the framework. The libraries we implemented and the programs we tested demonstrated both the validity of our framework and its relevance for applying metamorphic testing in real-world scenarios. However, crafting these relations to verify that a program behaves according to its specification in the context of natural language processing requires a strong understanding of the available tools, especially when working with LLMs.

We aim to propose additional input and output patterns in order to make the framework more general and easier to use. We will keep the validation work in order to increase the user confidence.

One major challenge of the proposed patterns is the environmental cost associated with the use of LLMs. Future work will also focus on reducing the reliance on LLMs in both the generation of metamorphic relations and the comparison of outputs. Advancements in semantic analysis methods, enabling better understanding and controlled modification of sentence meaning, will be essential moving forward.

[1]https://gricad-gitlab.univ-grenoble-alpes.fr/dubousql/mtf

	Nb. of tests	Pass	Fail
Multilangual library			
Word_addition	20	20	0
Word_subtraction	20	20	0
Word_switch	20	20	0
Word_substitution _synonym_ChatGPT	20	7	13
Paraphrase_long_ChatGPT	10	6	4
Paraphrase_short_ChatGPT	3	2	1
Paraphrase_simple_ChatGPT	18	13	5
Change_locution_ChatGPT	14	9	5
Japanese Library			
LengthUp	10	9	1
LengthDown	10	10	0
AgeChild	5	5	0
AgeAdult	5	5	0
Syn	10	8	2

Figure 3: Manual testing of some input relations

Table 1: Similarity evaluation

	Mei-chan test cases	Edge test cases
Similarity with ChatGPT	0.96	0.85
Bert Similarity-Embedding	1	0.75

References

[1] Y. Cai, P. Liang, Y. Wang, Z. Li, and M. Shahin. 2025. Demystifying issues, causes and solutions in LLM open-source projects. *Journal of Systems and Software* 227 (2025), 112452. doi:10.1016/j.jss.2025.112452

[2] D. Cao, H. Guo, and C. Tao. 2021. MT4ImgRec: A Metamorphic Testing Tool for Image Recognition Software. In *The 33rd Int. Conf. on Software Engineering and Knowledge Engineering, (SEKE), KSIR Virtual Conference Center, USA, 2021*, Shi-Kuo Chang (Ed.). KSI Research Inc., 636. doi:10.18293/SEKE2021-205

[3] T. Y. Chen, S. C. Cheung, and S. M. Yiu. 2020. Metamorphic Testing: A New Approach for Generating Next Test Cases. arXiv:2002.12543 [cs.SE] https://arxiv.org/abs/2002.12543

[4] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu. 2025. An Empirical Study on Challenges for LLM Application Developers. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). doi:10.1145/3715007 Just Accepted.

[5] Y. Deng, X. Zheng, T. Zhang, H. Liu, G. Lou, M. Kim, and T. Y. Chen. 2023. A Declarative Metamorphic Testing Framework for Autonomous Driving. *IEEE*

Transactions on Software Engineering 49, 4 (2023), 1964–1982. doi:10.1109/TSE.2022.3206427

[6] A. Duque-Torres, D. Pfahl, C. Klammer, and S. Fischer. 2024. MetaExploreX: A Visualisation Tool for Selecting and Constraining Metamorphic Relations. In *IEEE Int. Conf. on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*. 219–222. doi:10.1109/SANER-C62648.2024.00036

[7] P. Gómez-Abajo, P. C. Cañizares, A. Núñez, E. Guerra, and J. de Lara. 2023. Gotten: A Model-Driven Solution to Engineer Domain-specific Metamorphic Testing Environments. In *ACM/IEEE Int. Conf. on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 65–69. doi:10.1109/MODELS-C59198.2023.00021

[8] T. Henry and S. Savourat. 2025. *MTF: Metamorphic Testing Framework*. https://gricad-gitlab.univ-grenoble-alpes.fr/dubousql/mtf

[9] H. Huang, Y. Pan, L. Luo, and Y. Wang. 2024. MTSA: Multi-Granularity Metamorphic Testing for Sentiment Analysis Systems. In *IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. 1334–1343. doi:10.1109/COMPSAC61105.2024.00176

[10] S. Hyun, M. Guo, and M. A. Babar. 2024. METAL: Metamorphic Testing Framework for Analyzing Large-Language Model Qualities. In *IEEE Conf. on Software Testing, Verification and Validation (ICST)*. 117–128. doi:10.1109/ICST60714.2024.00019

[11] N. Li, Y. Li, Y. Liu, L. Shi, K. Wang, and H. Wang. 2024. Drowzee: Metamorphic Testing for Fact-Conflicting Hallucination Detection in Large Language Models. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 336 (Oct. 2024), 30 pages. doi:10.1145/3689776

[12] Z. Li, J. Chen, H. Chen, L. Xu, and W. Guo. 2024. Detecting Bias in LLMs' Natural Language Inference Using Metamorphic Testing. In *IEEE 24th Int. Conf. on Software Quality, Reliability, and Security Companion (QRS-C)*. 31–37. doi:10.1109/QRS-C63300.2024.00015

[13] Z. Li, Z. Li, K. Xiao, and X. Li. 2023. Evaluating LLM's Code Reading Abilities in Big Data Contexts using Metamorphic Testing. In *9th Int. Conf. on Big Data and Information Analytics (BigDIA)*. 232–239. doi:10.1109/BigDIA60676.2023.10429345

[14] C. Miura, S. Chen, S. Saiki, M. Nakamura, and K. Yasuda. 2022. Assisting Personalized Healthcare of Elderly People: Developing a Rule-Based Virtual Caregiver System Using Mobile Chatbot. *Sensors* 22, 10 (2022). doi:10.3390/s22103829

[15] T. Nakata, S. Chen, S. Saiki, M. Nakamura, et al. 2023. Employing large language models for dialogue-based personalized needs extraction in smart services. *Computer Science & Information Technology (CS & IT)* 13, 24 (2023), 21–33.

[16] T. Nakata, M. Nakamura, S. Chen, and S. Saiki. 2024. Needs Companion: A Novel Approach to Continuous User Needs Sensing Using Virtual Agents and Large Language Models. *Sensors* 24, 21 (2024). doi:10.3390/s24216814

[17] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824. doi:10.1109/TSE.2016.2532875

[18] S. Speth and A. Pretschner. 2025. GeMTest: A General Metamorphic Testing Framework. In *47th IEEE/ACM Int. Conf. on Software Engineering, (ICSE) - Companion Proceedings, 2025*. IEEE, 41–44. doi:10.1109/ICSE-COMPANION66252.2025.00020

[19] G. Wang, Y. Li, Y. Liu, G. Deng, T. Li, G. Xu, Y. Liu, H. Wang, and K. Wang. 2024. MeTMaP: Metamorphic Testing for Detecting False Vector Matching Problems in LLM Augmented Generation. In *IEEE/ACM 1st Int. Conf. on AI Foundation Models and Software Engineering (Forge) Conference*. 12–23. doi:10.1145/3650105.3652297

[20] Z. Ying, D. Towey, A. Bellotti, C. Chua, and Z. Quan Zhou. 2025. Metamorphic Relation Patterns for Metamorphic Testing, Exploration and Robustness. *Software Testing, Verification and Reliability* 35, 2 (2025), e70003. doi:10.1002/stvr.70003 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.70003

[21] J. Yoon, R. Feldt, and S. Yoo. 2025. Adaptive Testing for LLM-Based Applications: A Diversity-Based Approach. In *IEEE Int. Conf. on Software Testing, Verification and Validation Workshops (ICSTW)*. 375–382. doi:10.1109/ICSTW64639.2025.10962467

[22] H. Zhu. 2015. JFuzz: A Tool for Automated Java Unit Testing Based on Data Mutation and Metamorphic Testing Methods. In *2nd Int. Conf. on Trustworthy Systems and Their Applications*. 8–15. doi:10.1109/TSA.2015.13

Appendix

Input patterns for Multilingual library. [12, 20]

(1) **word_addition**: select a random word from the input and insert it again at a random location in the input. (duplication)
(2) **word_addition_mBERT**: add a word at a random location in the input
(3) **word_subtraction**: remove a random word from the input
(4) **word_switch**: switch a random word with another one
(5) **word_substitution_synonym_ChatGPT**: replace a random word by a synonym using ChatGPT
(6) **word_substitution_synonym_SBERT**: replace a random word by a synonym using WordNet databse and SBERT
(7) **word_substitution_synonym_NLTK**: replace a random word by a synonym using WordNet database
(8) **word_substitution_synonym_mBERT**: replace a random word by a synonym using mBERT
(9) **paraphrase_long_ChatGPT**: longer reformulation of the input using ChatGPT
(10) **paraphrase_short_ChatGPT**: shorter reformulation of the input using ChatGPT
(11) **paraphrase_simple_ChatGPT**: normal reformulation of the input using ChatGPT
(12) **change_locution_ChatGPT**: change age, sex, ethny, or social status in the input using ChatGPT

Input patterns for Japanese text. [12, 20]

(1) **Syn**: replace a word by a synonym, specific to the Japanese language, using ChatGPT
(2) **lenUP**: longer reformulation of the input, specific to the Japanese language, using ChatGPT
(3) **lenDOWN**: shorter reformulation of the input, specific to the Japanese language, using ChatGPT using ChatGPT
(4) remove_characters: remove a random character from the input
(5) **genderChange**: reformulation of the input by changing the gender, specific to the Japanese language, using ChatGPT
(6) **ageChild**: reformulation of the input like a child, specific to the Japanese language, using ChatGPT
(7) **ageAdult**: reformulation of the input like an adult, specific to the Japanese language, using ChatGPT

Input patterns dedicated to Mei-chan.

(1) **remove_when**: Remove temporal information from the input using ChatGPT ("when" field of 6W1H format)
(2) **remove_where**: Remove location information from the input using ChatGPT ("where" field of 6W1H format)
(3) **remove_who**: Remove information about the performer of the action from the input using ChatGPT ("who" field of 6W1H format)
(4) **remove_whom**: Remove information about the target of the action from the input using ChatGPT ("whom" field of 6W1H format)
(5) **remove_why**: Remove excedent informations about the action from the input using ChatGPT ("why" field of 6W1H format)
(6) **remove_what**: Remove action informations from the input using ChatGPT ("what" field of 6W1H format)
(7) **remove_how**: Remove informations about how to perform the action from the input using ChatGPT ("how" field of 6W1H format)
(8) **add_when**: add temporal information from the input using ChatGPT ("when" field of 6W1H format)
(9) **add_where**: add location information from the input using ChatGPT ("where" field of 6W1H format)
(10) **add_who**: add information about the performer of the action from the input using ChatGPT ("who" field of 6W1H format)
(11) **add_whom**: add information about the target of the action from the input using ChatGPT ("whom" field of 6W1H format)
(12) **add_why**: add excedent informations about the action from the input using ChatGPT ("why" field of 6W1H format)
(13) **add_what**: add action informations from the input using ChatGPT ("what" field of 6W1H format)
(14) **add_how**: add informations about how to perform the action from the input using ChatGPT ("how" field of 6W1H format).

Output patterns.

(1) **LLM_metric**: Using a Large Language Model (ChatGPT from OpenAI) to compare the evaluated output to the reference one.
(2) **BERT_metric**: Using SBERT model to compare the embeddings of the tokenized evaluated output and reference one.
(3) **statistical_metric**: Using basic Natural Language Processing techniques to compare the evaluated output to the reference one. Currently a simple word matching percentage.